🐳☕ Copying JAR into Docker Container & Creating Image

+ Automating Image Creation with Dockerfile – Creative Theory Notes

---

PART A: Manual Image Creation Using Docker Commands

---

1. Inspecting the Running JDK Container

The speaker starts with a **running Docker container** that already contains a JDK.

Key actions:

- Stops the running Spring Boot JAR process
- Opens a terminal
- Lists running Docker processes
- Uses `docker exec` to access the container shell

Inside the container, the filesystem is explored:

- JDK-related directories are inspected
- The `/temp` directory is chosen as a location to store the application JAR

This step helps understand **where and how files live inside a container**.

---

2. Copying JAR File into the Container

To move the Spring Boot application into the container, the speaker uses **file copy from host to container**.

Process:

- The JAR file is copied from the host machine
- Destination: `/temp` directory inside the container
- File transfer is verified by listing directory contents

This prepares the container to run the Spring Boot application internally.

---

3. Creating a New Image from the Modified Container

After modifying the container (by adding the JAR), the speaker creates a **new Docker image** using the container's current state.

Key concept:

- A container can be converted into an image
- The image is named and tagged for identification

When the image is run:

- It starts **JShell by default**, because that is the base image's CMD

This reveals an important limitation:
👉 **The default startup command must be customized.**

---

4. Customizing the Default Command (CMD)

To ensure the container runs the Spring Boot app instead of JShell, the speaker updates the image creation process.

Solution:

- Use `docker commit` with the `--change` option
- Override the default CMD
- Specify `java -jar` as the startup command

Important detail:

- The command must be passed as an **array of strings**
- This ensures Docker correctly interprets the execution command

Now, the new image runs the application automatically.

---

5. Running the Image and Networking Issue

When the updated image is run:

- The Spring Boot app starts successfully
- It runs inside the container on **port 8081**

However:

- The application is **not accessible from the host browser**

Reason:

- Docker containers have **isolated networking**
- Container ports are not exposed by default

---

6. Port Mapping with `-p` Option

To make the application accessible, **port mapping** is used.

Concept:

- Host port ↔ Container port

By running the container with `-p`:

- Container's port 8081 is mapped to the host
- The application becomes accessible via browser

This demonstrates how Docker safely isolates applications while still allowing controlled access.

---

7. Portability Advantage of Docker Images

Once the image is created:

- It can be pushed to **Docker Hub**
- It can run on **any machine with Docker**
- No need to install Java or dependencies manually

This highlights Docker's core benefit:
👉 **Build once, run anywhere**

---

8. Limitation of the Manual Approach

The speaker reviews the manual steps:

1. Start a container
2. Copy the JAR file
3. Commit the container
4. Modify CMD
5. Run with port mapping

Problems:

- Time-consuming
- Error-prone
- Not scalable
- Difficult for frequent changes

This motivates the need for **automation**.

PART B: Automating Image Creation with Dockerfile

---

9. Introduction to Dockerfile

A **Dockerfile** is a text file that contains instructions to build a Docker image automatically.

Creation:

- Can be created via IDE plugin
- Or as a normal file named `Dockerfile`

Dockerfile eliminates repetitive manual steps.

---

10. Base Image Selection (`FROM`)

The first instruction in a Dockerfile defines the **base image**.

Example:

- OpenJDK 22 image

This base image already contains:

- Operating system
- Java runtime and tools

All other instructions build **on top of this base layer**.

---

11. Adding the JAR File (`ADD` / `COPY`)

The next step is copying the application JAR into the image.

Details:

- Source: `target` folder (host system)
- Destination: Inside the container

This embeds the Spring Boot application directly into the image.

---

12. Defining Startup Command (`ENTRYPOINT`)

To ensure the container runs the app automatically:

- `ENTRYPOINT` is used
- Specifies `java -jar` command
- Executes the application when the container starts

Difference:

- `CMD` → Can be overridden
- `ENTRYPOINT` → Fixed execution command

This makes the image production-ready.

---

13. Building the Image Using Dockerfile

The image is built using:

- `docker build` command
- `-t` flag to tag the image
- `.` (dot) specifies the current directory as build context

Docker reads:

- Dockerfile
- Application files
- Metadata from base image

And produces a new image automatically.

---

14. Build Context and `.dockerignore`

Docker sends the build context to the daemon.

To optimize this:

- `.dockerignore` file is used
- Works like `.gitignore`
- Excludes unnecessary files from build

This improves:

- Build speed
- Image cleanliness

---

15. Docker Build Cache Mechanism

Docker uses **layer caching** to optimize builds.

Key idea:

- If a layer hasn't changed, Docker reuses it
- Only changed layers are rebuilt

Benefits:

- Faster builds
- Efficient reuse of base images

This is especially useful during development.

---

16. Verifying Image Creation

After the build completes:

- The image is exported
- Listed using Docker image commands
- Ready to run with port mapping

This confirms successful automation.

---

17. Conclusion

This section demonstrates two approaches to Docker image creation:

- **Manual approach** → Educational but inefficient
- **Dockerfile automation** → Scalable, repeatable, professional

Dockerfile is the **industry-standard way** to build images and is essential for:

- CI/CD pipelines
- Microservices
- Multi-container applications

This completes the transition from **local Spring Boot app** to a **fully containerized application** 🚀🐳

If you want next, I can:

- Write the **complete Dockerfile (final version)**
- Explain **CMD vs ENTRYPOINT** deeply
- Show **Spring Boot + Docker best practices**
- Introduce **Docker Compose**
- Convert everything into **exam notes / lab manual**